Jenkins CI/CD on AWS EC2

*Hands-on Project Documentation & Troubleshooting (*manually triggered*)*

1. Project objective

Build a simple CI/CD pipeline where Jenkins checks out an HTML file from GitHub, validates it, and deploys it to an Apache web server on a separate EC2 instance using SSH/SCP.

2. Architecture

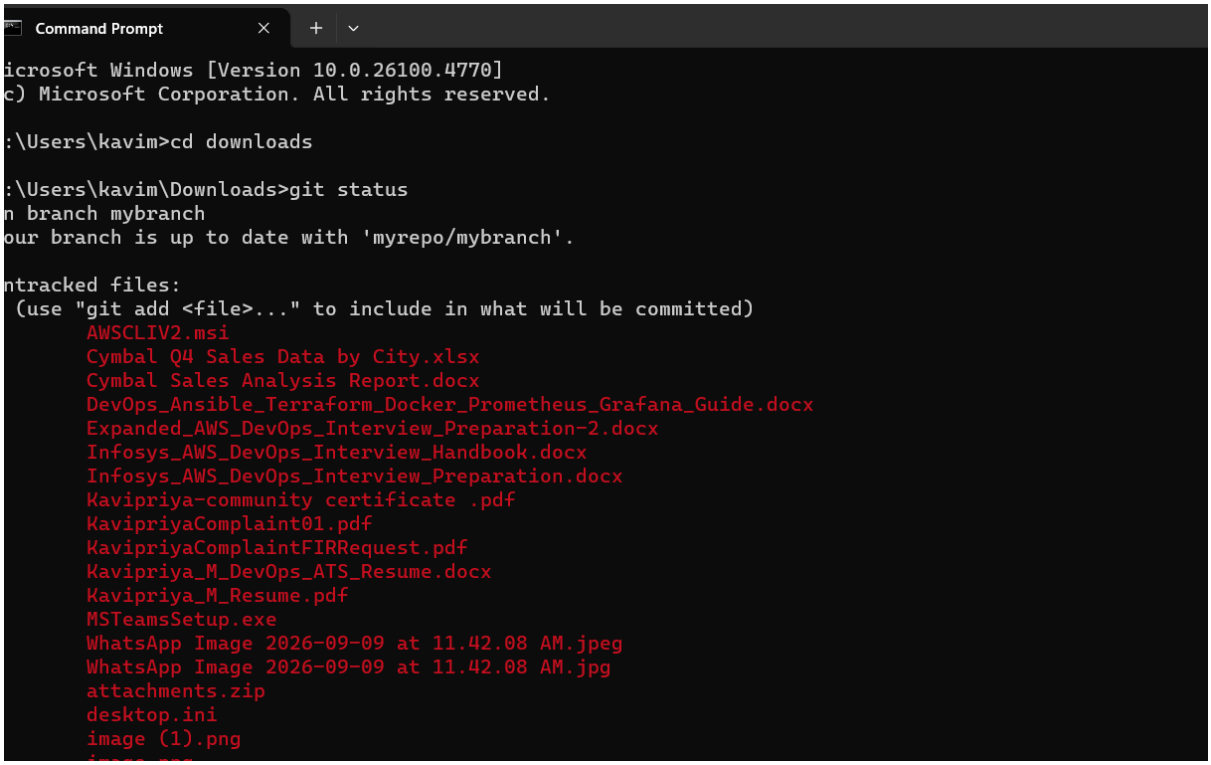
Component	Role	Lab private IP (example)
Server 1 — Apache	Hosts the website; receives deployment file	172.31.23.140
Server 2 — Jenkins	Runs the pipeline and copies the file over SSH	172.31.20.218
GitHub	Stores Jenkinsfile and welcome.html	Repository URL configured in Jenkins

The private IPs above came from this lab and are examples only. Replace them if the instances are recreated or addresses change.

3. Pipeline flow

1. Checkout: Jenkins pulls the repository from GitHub.
2. Build / Validate: shell checks confirm that welcome.html exists.
3. Deploy: sshagent loads the Jenkins SSH credential and scp transfers the HTML file to Apache.
4. Verification: open the deployed page through the Apache server's public IP and HTTP port 80, after the file is in the configured Apache document root.

Cmd:



```
C:\Users\kavim\Downloads>git push myrepo mybranch
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 330 bytes | 330.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/kavima101-netizen/myrepo.git
   b00024b..c7f7f9a  mybranch -> mybranch

C:\Users\kavim\Downloads>git  add jenkinsfile

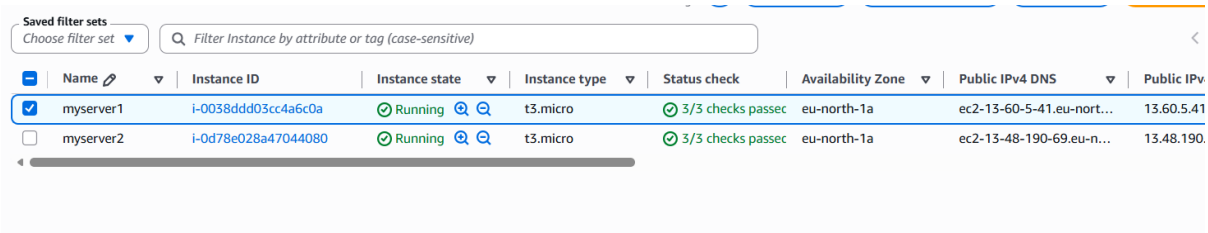
C:\Users\kavim\Downloads>git commit -m "jenkin file added5"
[mybranch 5533f93] jenkin file added5
 1 file changed, 1 insertion(+), 1 deletion(-)

C:\Users\kavim\Downloads>git push myrepo mybranch
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 337 bytes | 337.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/kavima101-netizen/myrepo.git
   c7f7f9a..5533f93  mybranch -> mybranch

C:\Users\kavim\Downloads>
```

4. Prerequisites

- Two running EC2 instances with network routes that allow private communication.
- Jenkins installed on Server 2 and Apache/httpd installed and running on Server 1.
- Git, Pipeline, and SSH Agent plugin support in Jenkins.
- Jenkins credential of kind SSH Username with private key; username must match the Apache OS image (commonly ec2-user on Amazon Linux).
- Security groups allow TCP/22 from the Jenkins security group to Apache. HTTP TCP/80 is allowed only from intended viewers.
- Repository contains Jenkinsfile and welcome.html.



5. Example Jenkinsfile

This example transfers to /tmp for a controlled transfer test. To publish the page, use a controlled deployment step to place it in Apache's actual document root (commonly /var/www/html).

```
pipeline {

    agent any
```

```
stages {
    stage('Checkout') {
        steps {
            echo 'Downloading code from GitHub'
            git branch: 'mybranch',
                url: 'https://github.com/kavima101-netizen/myrepo.git'
        }
    }
    stage('Build') {
        steps {
            echo 'Checking HTML file'
            sh 'test -f welcome.html'
        }
    }
    stage('Deploy') {
        steps {
            echo 'Deploying HTML to Apache server'
            sshagent(credentials: ['server1-sshkey']) {
                sh '''
                    scp -o StrictHostKeyChecking=yes \
                    welcome.html ec2-user@172.31.23.140:/tmp/welcome.html

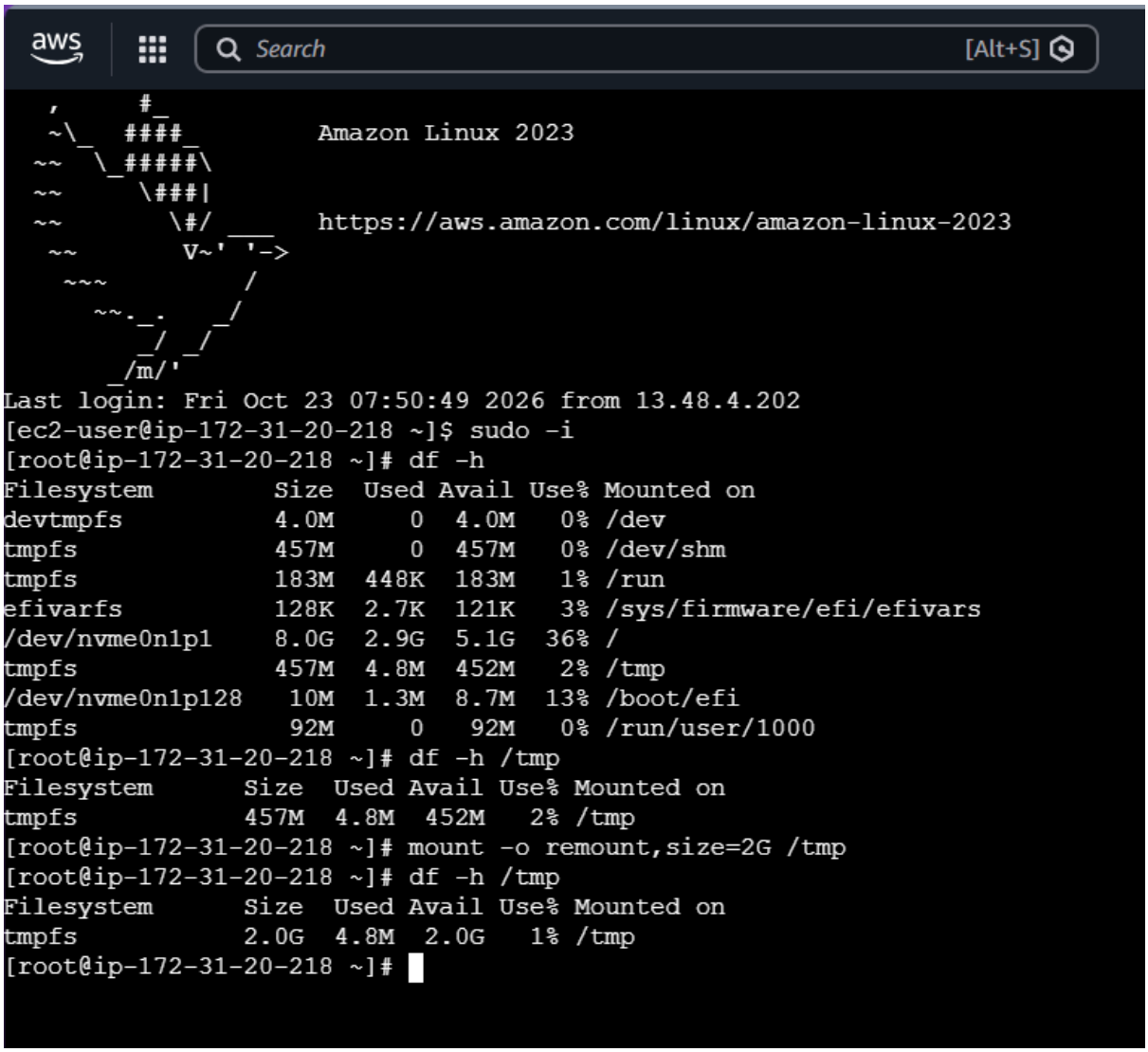

                    ssh -o StrictHostKeyChecking=yes \
                    ec2-user@172.31.23.140 \
                    'sudo install -m 644 /tmp/welcome.html /var/www/html/welcome.html'
                '''
            }
        }
    }
}
```

}

6. Issues encountered and fixes

Jenkins temporary disk-space warning

Checked filesystem usage. Verify the exact path mentioned in the warning using df -h <path> and df -i. Expanding a different mount does not fix the filesystem that Jenkins is checking.



No such DSL method 'sshagent'

Jenkins did not recognize the sshagent pipeline step. Install/enable the SSH Agent plugin and restart Jenkins if requested.

Malformed SCP destination / No such file or directory

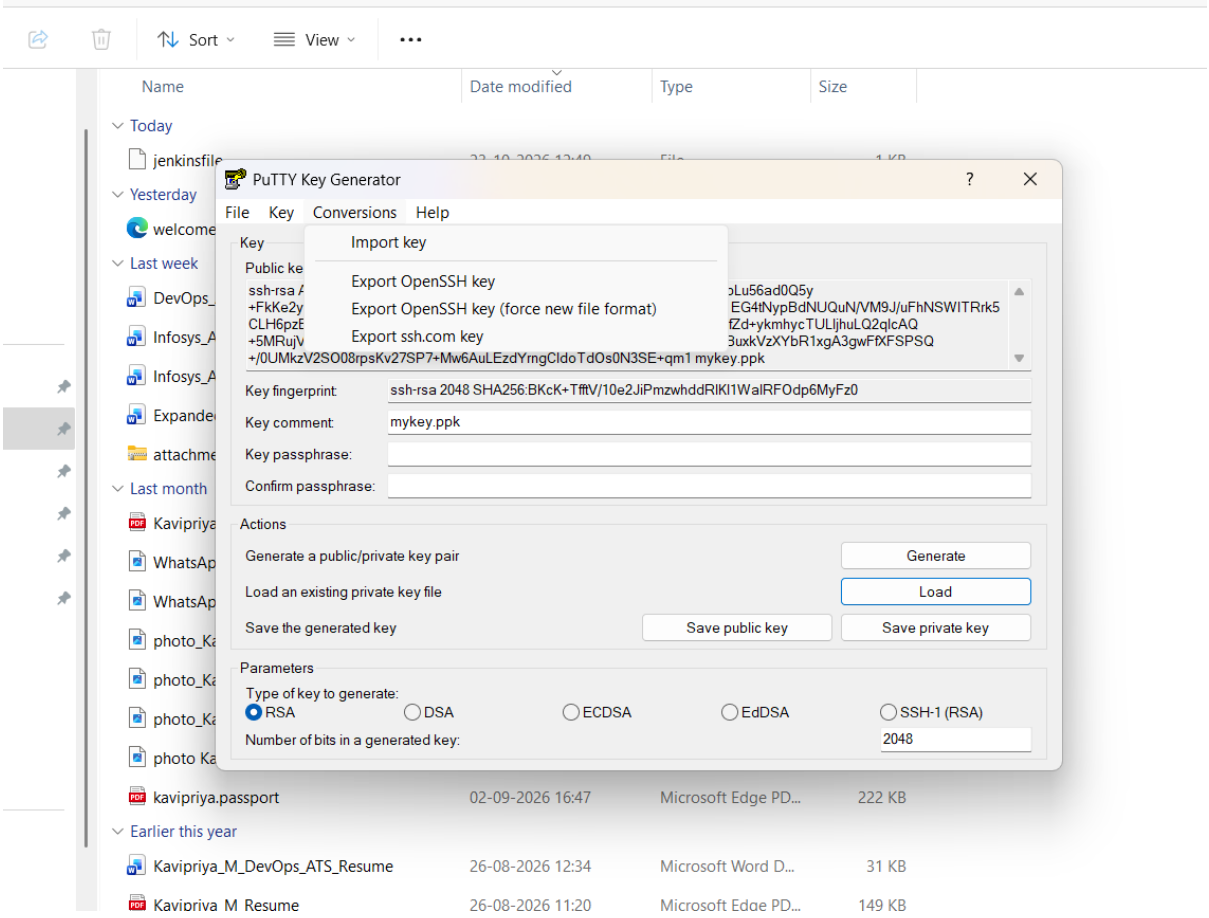
A space before the colon made the SCP destination invalid. Correct syntax: scp welcome.html ec2-user@172.31.23.140:/tmp/welcome.html — no space between host and colon.

Host key not known / Host key verification failed

The Jenkins OS user's SSH client did not have a trusted host-key entry for the Apache host. Confirm the Jenkins home directory with getent passwd jenkins and ensure known_hosts is in that user's actual .ssh directory.

Ping failed but SSH test passed

ICMP may be blocked. A successful TCP/22 test proves port reachability, not successful SSH login.



Permission denied (publickey)

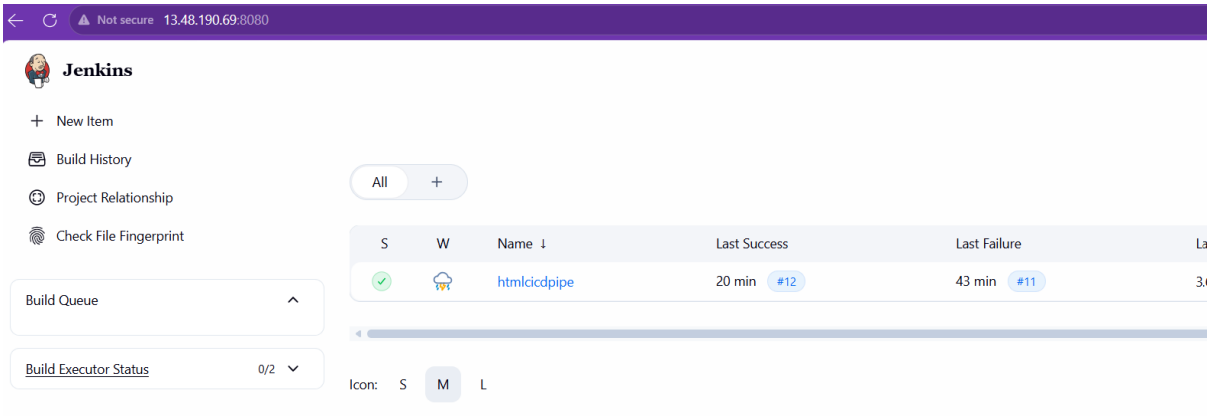
The host was reachable, but authentication failed. Check Jenkins SSH credential ID, username, matching private key, and the target user's authorized_keys. Do not put private keys in GitHub.

script returned exit code 1 or 255

These are generic failure codes. Read the first meaningful error above the final line in Console Output.

Deployment verification

Confirm the file is in Apache's configured document root, Apache is running, permissions are correct, and TCP/80 is allowed from the client. A file in /tmp is not automatically served by Apache.



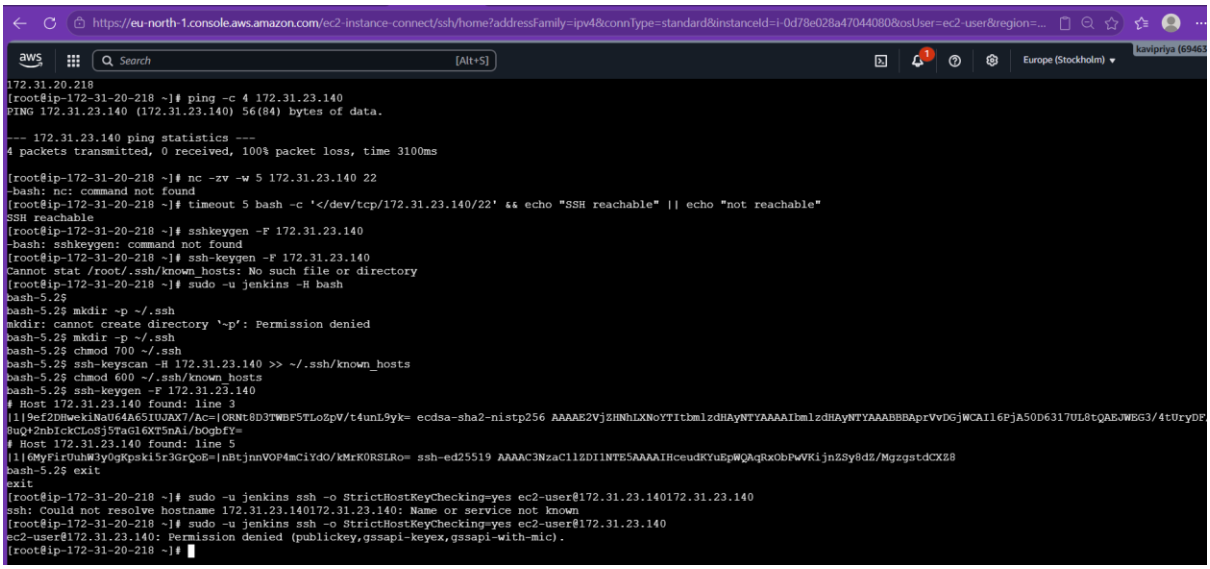
7. Useful diagnostic commands

Check disk usage

```
df -h
 df -i
```

Check SSH port from Jenkins

```
nc -zv -w 5 172.31.23.140 22
```



Fallback TCP test

```
timeout 5 bash -c '</dev/tcp/172.31.23.140/22' && echo 'SSH port reachable' || echo 'SSH port not reachable'
```

Find a host key entry

```
sudo -u jenkins ssh-keygen -F 172.31.23.140 -f /var/lib/jenkins/.ssh/known_hosts
```

Check Apache service (Amazon Linux)

```
sudo systemctl status httpd
```

8. Security notes

- Never commit PEM files, private keys, passwords, tokens, or other secrets to the repository.
- Verify the SSH host-key fingerprint through a trusted channel before trusting it. ssh-keyscan alone does not authenticate a server.

- Keep StrictHostKeyChecking=yes; avoid disabling host-key checking as a permanent fix.
- Restrict SSH to the Jenkins security group and restrict Jenkins UI access to trusted administrators.
- Do not make the entire Apache document root world-writable. Use a controlled deployment mechanism and least privilege.

9. Suggested GitHub repository layout

```
jenkins-html-cicd-handson/
├── Jenkinsfile
├── welcome.html
├── README.md
└── docs/
    └── troubleshooting.md
```

10. Completion checklist

☐ Jenkins checks out the repository successfully.

☐ Build / Validate stage confirms welcome.html exists.

☐ SSH Agent step loads the intended Jenkins credential.

☐ SCP transfer succeeds to the Apache instance.

☐ File is placed in the configured Apache document root.

☐ Website loads at http://<APACHE_PUBLIC_IP>/welcome.html.

```
SSH_AUTH_SOCK=/tmp/ssh-XXXXXXQs8Im2/agent.80273
SSH_AGENT_PID=80277
Running ssh-add (command line suppressed)
[ssh-agent] Started.
[Pipeline] {
[Pipeline] sh
+ scp -o StrictHostKeyChecking=yes welcome.html ec2-user@172.31.23.140:/tmp/welcome.html
+ ssh -o StrictHostKeyChecking=yes ec2-user@172.31.23.140 'sudo install -m 644 /tmp/welcome.html /var/www/html/welcome.html'
[Pipeline] }
$ ssh-agent -k
unset SSH_AUTH_SOCK;
unset SSH_AGENT_PID;
echo Agent pid 80277 killed;
[ssh-agent] Stopped.
[Pipeline] // sshagent
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

15. Project summary

“I built a basic CI/CD pipeline for a static HTML website using GitHub, Jenkins, and Apache on two Linux EC2 instances. I stored the HTML and Jenkinsfile in GitHub. Jenkins checked out the repository, validated that the HTML file existed, and deployed it to the Apache server using SSH/SCP. I verified the deployment by opening the page in a browser and tested an update by pushing a new commit.”